

Day 7: Quantum Machine Learning

SC Quantathon v3 Bootcamp
Clemson Quantum Club

Quantum machine learning, in the form that runs today, is Day 6's variational loop pointed at data: a circuit that depends on an input x and on trainable angles $\vec{\theta}$, an expectation value read off it, and a classical optimizer. These notes cover the two models that account for nearly all of it, the quantum kernel and the variational classifier, the encoding step both depend on, the gradient that trains the second, and the obstacle that limits both. They follow the order of the Day 7 notebook. Both datasets are synthetic and two-dimensional (Figure 1) so that every decision boundary can be drawn.

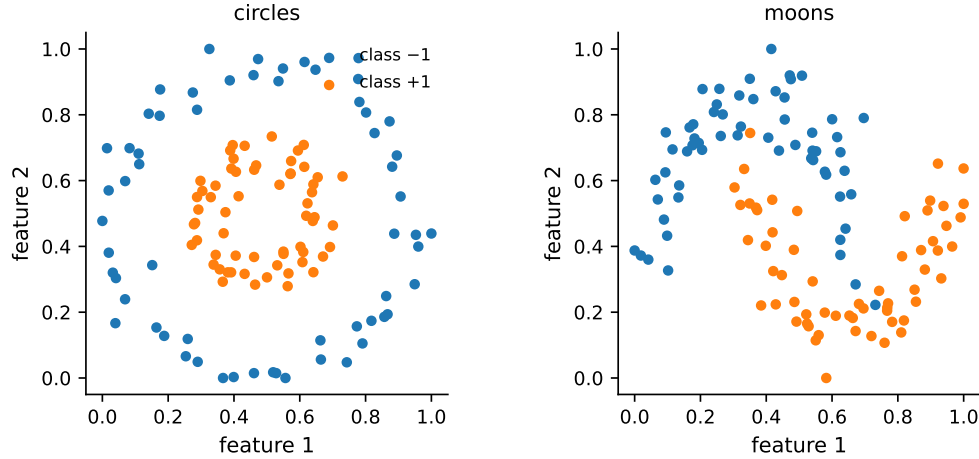


Figure 1: The two datasets, 120 points each, features scaled to $[0, 1]$: concentric circles and interleaved moons. Neither is separable by a straight line.

1 Encoding classical data into qubits

A data point is a list of numbers; a qubit register is a state vector. The circuit that maps one to the other is the *feature map* $U(x)$, and the state it prepares,

$$|\phi(x)\rangle = U(x) |0 \cdots 0\rangle, \quad (1)$$

is the only way the data ever enter a model. Three encodings cover most of what is used. *Basis encoding* sends a bitstring to the basis state that spells it, one qubit per bit and no superposition. *Angle encoding* sends each feature $x_i \in [0, 1]$ to a rotation, $|\phi(x)\rangle = \bigotimes_i R_y(\pi x_i) |0\rangle$, one qubit per feature and a product state. *Amplitude encoding* sends a vector of 2^n numbers, normalized, to the amplitudes of an n -qubit state: exponentially compact, but preparing a general such state

costs a circuit of depth of order 2^n . The notebook encodes $(3, 1, 2, 4)$ this way and reads back the probabilities $(0.300, 0.033, 0.133, 0.533)$, which are the squares of the normalized entries.

A product-state encoding can be simulated one feature at a time, so any kernel built from it is classically easy. Concretely, for the angle map the overlap factorizes: $\langle 0 | R_y(\pi x_i)^\dagger R_y(\pi z_i) | 0 \rangle = \langle 0 | R_y(\pi(z_i - x_i)) | 0 \rangle = \cos \frac{\pi(z_i - x_i)}{2}$ on each qubit, so

$$k_{\text{angle}}(x, z) = \prod_i \cos^2 \frac{\pi(x_i - z_i)}{2}, \quad (2)$$

a product of one factor per feature that a classical computer evaluates in time linear in the number of features, with no state vector anywhere. For the notebook's two test points, $(0.3, 0.8)$ and $(0.9, 0.1)$, Eq. (2) gives 0.071, while the entangling map gives 0.105: the CNOT and the product rotation have changed the geometry of the feature space, and no per-feature formula reproduces it. The maps used in practice add entangling gates and products of features. The notebook's *entangling map* (Figure 2) encodes each feature as an R_y angle, applies a CNOT, and then rotates the second qubit by $\pi x_0 x_1$, so the state depends on the features jointly. The *ZZ feature map* of Havlíček et al. is the best-known alternative: H on every qubit, a phase $2x_i$ on each, a phase $2(\pi - x_0)(\pi - x_1)$ on the pair through two CNOTs, the block repeated twice. Both are compared in the next section.

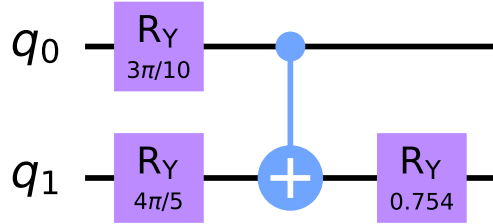


Figure 2: The entangling map for the point $(0.3, 0.8)$.

2 Quantum kernels and the support vector machine

Many classifiers use the data only through inner products between pairs of points. A *kernel* $k(x, z)$ is such an inner product in some feature space, and a support vector machine (SVM) finds the boundary that separates the classes with the widest margin in that space, using only the kernel matrix $K_{ij} = k(x_i, x_j)$. A *quantum kernel* is the overlap of two encoded states,

$$k(x, z) = |\langle \phi(x) | \phi(z) \rangle|^2 = |\langle 0 | U(x)^\dagger U(z) | 0 \rangle|^2, \quad (3)$$

which a processor estimates by running $U(x)^\dagger U(z)$ and counting how often all zeros comes back. Three properties make Eq. (3) a valid kernel. It is symmetric, and it equals 1 on the diagonal. And it is an inner product: $|\langle a | b \rangle|^2 = \text{Tr}(|a\rangle\langle a| |b\rangle\langle b|)$, so k is the inner product of the density matrices $|\phi(x)\rangle\langle\phi(x)|$, and any matrix of inner products is positive semidefinite, which is what the SVM's optimization needs.

The SVM itself never sees the feature space. Its training problem depends on the data only through K , and its decision function is

$$f(x) = \sum_i \alpha_i y_i k(x, x_i) + b, \quad (4)$$

a weighted vote of the training points, with the weights α_i found by a convex optimization and nonzero only for the *support vectors* that sit near the boundary. This is the *kernel trick*: any feature map, including one into a 2^n -dimensional Hilbert space, costs the same as long as its kernel can be evaluated. On hardware that cost is one circuit per pair, $N(N-1)/2 = 3160$ distinct circuits for the 80 training points and another $40 \times 80 = 3200$ for the test set, each needing enough shots to resolve overlaps near zero. Scikit-learn’s `SVC(kernel="precomputed")` takes the training kernel matrix instead of raw features. Table 1 compares the quantum kernel SVM on both datasets with a linear SVM and an RBF-kernel SVM on the raw features, 80 training and 40 test points each; Figure 3 draws the decision regions.

dataset	quantum kernel, train	quantum kernel, test	linear SVM, test	RBF SVM, test
circles	100%	100%	52%	100%
moons	85%	90%	88%	100%

Table 1: Test accuracy of the quantum kernel SVM with the entangling map against two classical baselines.

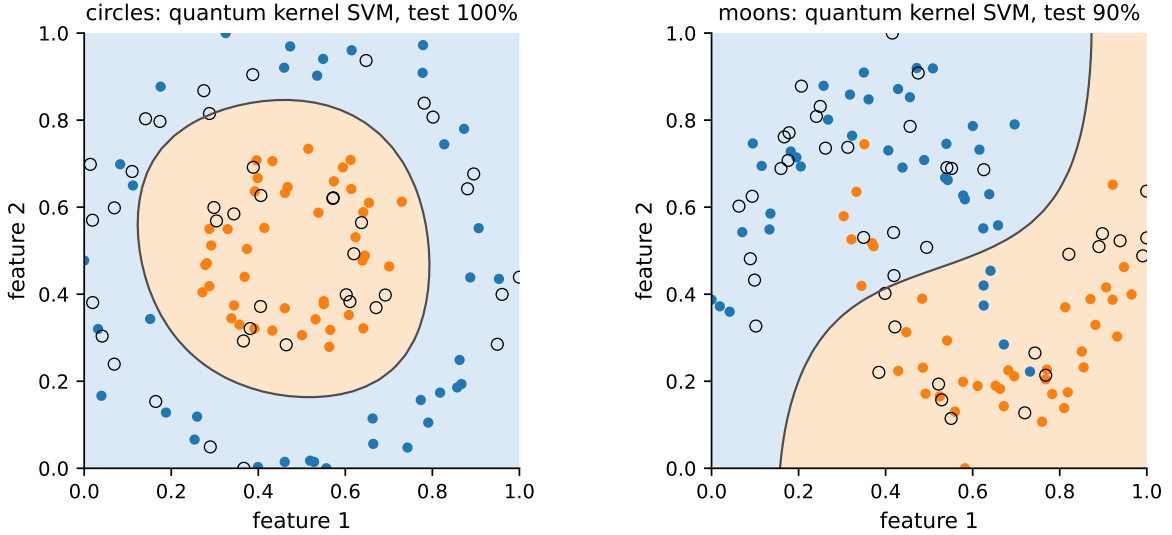


Figure 3: Decision regions of the quantum kernel SVM. Filled points are training data, open circles are test data.

Two things follow from the table. The RBF baseline does at least as well, so on data like this the quantum kernel earns nothing beyond a demonstration; the argument for quantum kernels rests on feature maps whose overlaps are hard to compute classically, and on data with matching structure. And the feature map decides everything. The same SVM with the ZZ feature map scores 78% on both datasets. Its pair phase $2(\pi - x_0)(\pi - x_1)$ runs from about 9 to 20 radians as the features cross $[0, 1]$, so two nearby points land on nearly orthogonal states and the kernel matrix loses the block structure the SVM needs (Figure 4): the entangling map’s within-class blocks average 0.45 and 0.80 against the ZZ map’s 0.39 and 0.74. Scaling the inputs by a factor $\beta < 1$ before encoding, the *kernel bandwidth*, is the usual fix. In the other direction, as qubit count grows, random feature maps push all overlaps toward zero, a phenomenon called *kernel concentration*, which makes K close to the identity and the SVM useless. Feature maps have to be matched to the data, not just made deep.

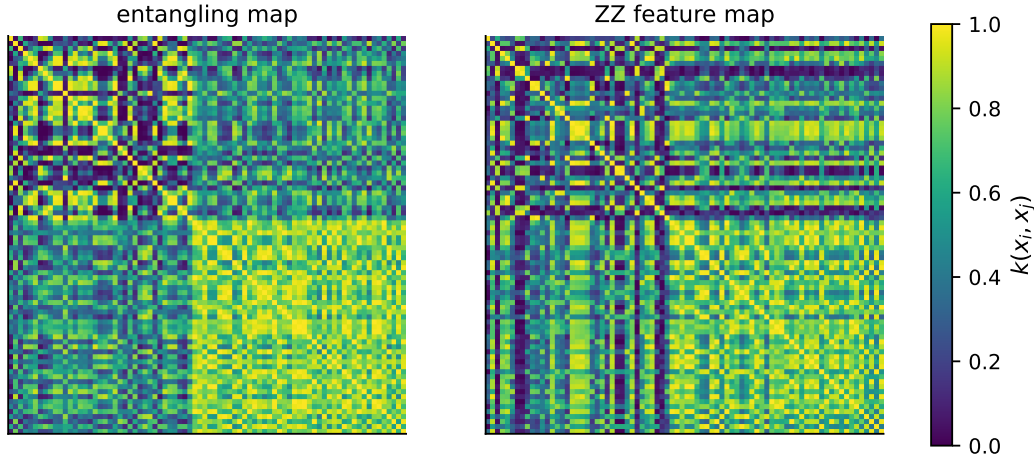


Figure 4: The 80×80 training kernel matrix of the circles data, rows and columns sorted by class, for the entangling map (left) and the ZZ feature map (right); the blocks are the two classes.

2.1 One kernel entry on the live machine

The notebook’s last section measures a single entry of Eq. (3) the way a processor has to: run $U(x)$ followed by $U(z)^\dagger$ and count how often 00 comes back. For the first two training points the exact overlap is 0.9206; the simulator’s 1000-shot estimate was 0.9250 and the recorded run on `ibm_kingston`, physical qubits 55 and 59, two `cz` gates at depth 13, gave 0.9290 (929 of 00, 65 of 01, 2 of 10, 4 of 11). At $p \approx 0.92$ and 1000 shots the shot noise alone is $\sqrt{p(1-p)/N} \approx 0.009$, so both estimates sit within one standard deviation of the exact value, and this entry says nothing about gate error. That is expected: a percent of two-qubit gate error shifts $P(00)$ by less than a percent, below the shot noise of a 1000-shot estimate, and the recorded value even sits above the exact one, the opposite direction from what depolarizing noise pushes. Gate error shows itself on entries near 0, which noise lifts away from zero and which a kernel matrix has many more of than entries near 1. Estimating a full 80×80 matrix to that precision is 3160 circuits at a thousand shots each, the cost quoted above.

3 A variational quantum classifier

The second model replaces the SVM with a trainable circuit. Encode x with the feature map, apply the hardware-efficient ansatz $W(\vec{\theta})$ of Day 6 (two qubits, two entangling layers, six angles), and read one expectation value,

$$f(x; \vec{\theta}) = \langle \phi(x) | W(\vec{\theta})^\dagger Z_0 W(\vec{\theta}) | \phi(x) \rangle \in [-1, 1], \quad (5)$$

whose sign is the predicted class (Figure 5). Any observable with eigenvalues ± 1 would serve; Z_0 is the cheapest, one qubit read in the computational basis, and the parity $Z_0 Z_1$ is the usual alternative. There is no bias term, so the boundary is wherever f crosses zero; shifting that threshold is a one-parameter classical afterthought that the notebook leaves out. Training chooses $\vec{\theta}$ to make f close to the labels $y_i \in \{-1, +1\}$ under the squared-error loss

$$L(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N (f(x_i; \vec{\theta}) - y_i)^2. \quad (6)$$

Every input and every angle is a circuit parameter, so one parameterized circuit and one Estimator call evaluate f on the whole dataset. At the notebook’s random starting angles the loss is 2.0; a model that outputs $f = 0$ everywhere has loss exactly 1, so a random start can be worse than saying nothing.

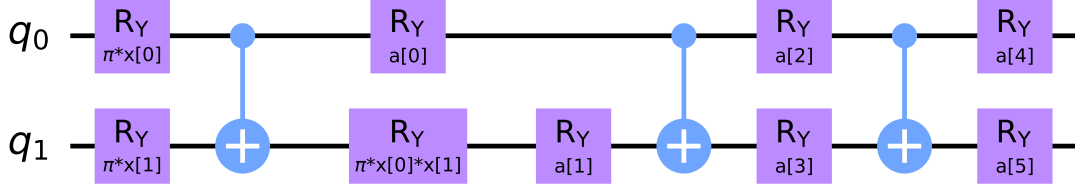


Figure 5: The classifier circuit: the entangling map with symbolic inputs x_0, x_1 , followed by the six-angle ansatz. The observable is Z on qubit 0.

4 Gradients and training

Every trainable gate is $R_y(\theta_k) = e^{-i\theta_k Y/2}$, so f is a sinusoid in each angle, $a + b \cos \theta_k + c \sin \theta_k$, exactly as for the energy on Day 6, and its derivative is the *parameter-shift rule*

$$\frac{\partial f}{\partial \theta_k} = \frac{1}{2} \left[f(\theta_k + \frac{\pi}{2}) - f(\theta_k - \frac{\pi}{2}) \right]. \quad (7)$$

Exact, not a finite difference, and made of ordinary circuit runs. Because f is a sinusoid of amplitude at most 1 in each angle, every component of its gradient is bounded by 1, and the loss landscape repeats with period 2π in every direction, so no learning rate can send an angle to infinity, only around the circle. The notebook checks the rule against a finite difference with step 10^{-5} and the two agree to 5×10^{-11} . The chain rule then gives the loss gradient,

$$\frac{\partial L}{\partial \theta_k} = \frac{2}{N} \sum_{i=1}^N (f(x_i; \vec{\theta}) - y_i) \frac{\partial f(x_i; \vec{\theta})}{\partial \theta_k}, \quad (8)$$

which costs $2P$ evaluations of the model, each an Estimator call over all N points, for P angles. Gradient descent, $\vec{\theta} \leftarrow \vec{\theta} - \eta \nabla L$ with $\eta = 0.4$ for 60 steps and the best of three random starts, brings the loss on the circles data from 0.64 after the first step to 0.44 and the training accuracy to 90%; all three starts end at the same loss, and the held-out accuracy is 78% (Figure 6). A six-angle circuit is a small model, and it draws a smooth closed curve around the inner class rather than the sharp ring of the kernel SVM. On hardware a single gradient step is $2PN$ circuit jobs before shot noise is considered, and that cost, not the mathematics, is what limits variational classifiers today.

5 Barren plateaus

Make the ansatz wider and deeper and something goes wrong that no optimizer can fix. For random angles in a sufficiently deep circuit, the gradient of any expectation value is zero on average and its *variance shrinks exponentially with the number of qubits*: for circuits deep enough to scramble the state like a random unitary, McClean et al. showed the variance is of order 2^{-n} for a Pauli

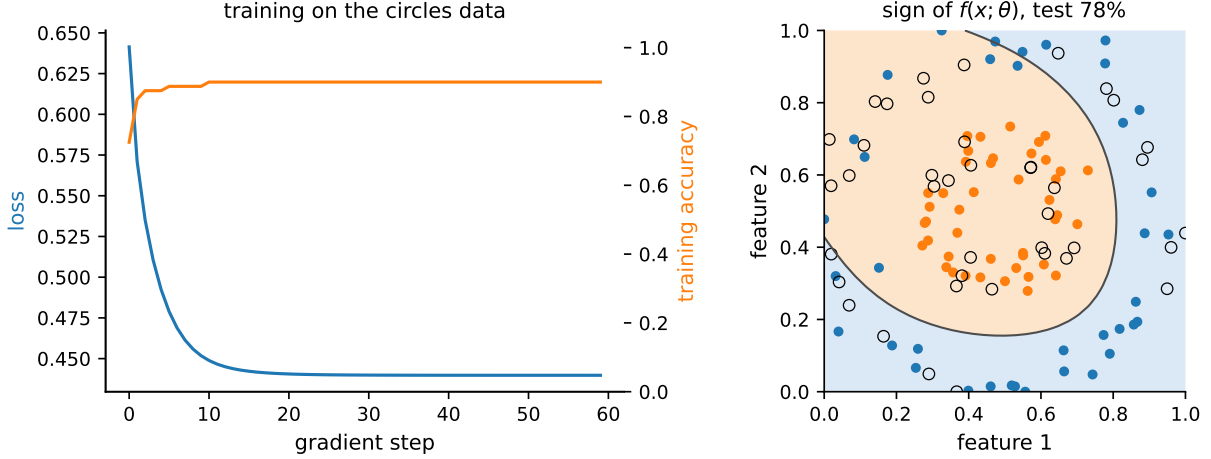


Figure 6: Left: loss and training accuracy over 60 gradient steps on the circles data. Right: the sign of $f(x; \theta)$ after training, with training points filled and test points open.

observable. The number comes from averaging over random states. For a state drawn uniformly from the $d = 2^n$ -dimensional Hilbert space, the expectation value of a traceless observable H with $H^2 = I$ has mean zero and

$$\text{Var} [\langle \psi | H | \psi \rangle] = \frac{\text{Tr } H^2}{d(d+1)} = \frac{1}{2^n + 1}, \quad (9)$$

since $\text{Tr } H^2 = d$. A parameter-shift gradient is half the difference of two such expectation values, so its variance is of the same order, and it halves with every added qubit. A landscape whose slopes are exponentially small everywhere cannot be climbed from a random start, since even the direction of the first step is lost in shot noise. The notebook measures this directly: a hardware-efficient ansatz with as many entangling layers as qubits, a hundred random parameter settings, and the parameter-shift derivative of $\langle Z_0 Z_1 \rangle$ with respect to the first angle (Table 2, Figure 7).

qubits n	2	3	4	5	6	8
$\text{Var}[\partial_{\theta_1} \langle Z_0 Z_1 \rangle]$	0.233	0.102	0.073	0.038	0.026	0.013

Table 2: Variance of one gradient component over 100 random parameter settings, for an ansatz with n qubits and n entangling layers.

The straight line on the log axis is the exponential: a factor of 0.62 per added qubit over this range, a factor of about 18 between 2 and 8 qubits. Eq. (9) sets the scale a fully scrambling circuit would reach: 0.200, 0.111, 0.059, 0.030, 0.015, and 0.004 for the six widths, for the expectation value itself; the gradient, half a difference of two such values, is of the same order. From four qubits on the measured variances sit above that scale, and the gap widens with width, because n entangling layers on n qubits is not yet deep enough to scramble the state completely. Deeper circuits approach the $1/(2^n + 1)$ line, and the plateau only gets flatter. The ways around it are an active research area: shallow circuits, ansätze built from the problem rather than from convenience (Day 6’s one-parameter ansatz is the model case), initializing near the identity, and structured feature maps. They all amount to not searching the whole exponentially large space at once.

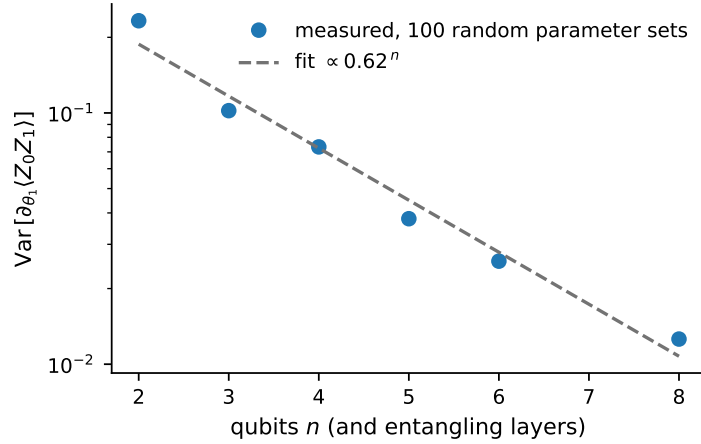


Figure 7: The variance of Table 2 on a log scale, with an exponential fit.

6 What transfers to the hackathon

- Fit the classical baselines first. A linear model, an RBF SVM, and a small tree ensemble take minutes and set the bar a quantum model has to clear; a result that does not beat them is not a result.
- Count features. A quantum model needs one qubit per feature for angle encoding and $\log_2$ of the count for amplitude encoding; a dataset with fifty columns has to be reduced, by principal components or by choosing columns, before it fits.
- Split and cross-validate. Report held-out accuracy, and repeat over several splits, since with 40 test points one point is 2.5%. Scale the features with statistics of the training set only and apply the same transformation to the test set; fitting the scaler on everything leaks the test data into the model.
- Watch the kernel matrix. If it is close to the identity, the map is too wide or too deep for the data (kernel concentration); if it is nearly all ones, the map is too narrow. Scale the inputs until the blocks appear.
- Budget shots. A kernel matrix for N training points is $N(N-1)/2$ circuits on hardware, each needing enough shots to resolve overlaps near zero; a variational classifier is $2PN$ circuits per gradient step. Simulate first and go to hardware for one confirming run.

7 Summary

- A feature map $U(x)$ turns data into a state, Eq. (1). Basis, angle, and amplitude encoding are the basic choices; entangling maps are what give quantum models a feature space a product state does not have.
- A quantum kernel is the overlap of two encoded states, Eq. (3): symmetric, unit diagonal, positive semidefinite. With an SVM it separates the circles and moons data (Table 1); an RBF kernel does as well, and the map’s bandwidth decides whether the kernel carries structure at all.

- A variational classifier reads the sign of an expectation value, Eq. (5), and is trained on the squared loss, Eq. (6), with parameter-shift gradients, Eqs. (7) and (8). Six angles reach 90% training accuracy on the circles data.
- The variance of a gradient component falls exponentially with qubit count in deep random circuits, a barren plateau (Table 2); shallow, problem-shaped circuits and careful initialization are the ways around it.
- One kernel entry measured on a processor landed within shot noise of the exact overlap; the cost of a whole matrix, thousands of such circuits, is the real constraint.
- Classical baselines, feature counts, held-out evaluation, the kernel matrix, and shot budgets decide whether a quantum model is worth running on a hackathon dataset.

References

- [1] V. Havlíček et al., “Supervised learning with quantum-enhanced feature spaces,” *Nature* **567**, 209 (2019). The ZZ feature map and the quantum kernel SVM.
- [2] M. Schuld and N. Killoran, “Quantum machine learning in feature Hilbert spaces,” *Phys. Rev. Lett.* **122**, 040504 (2019). Feature maps as kernels.
- [3] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, “Quantum circuit learning,” *Phys. Rev. A* **98**, 032309 (2018). The variational classifier and the parameter-shift rule.
- [4] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren plateaus in quantum neural network training landscapes,” *Nature Communications* **9**, 4812 (2018).
- [5] R. Shaydulin and S. M. Wild, “Importance of kernel bandwidth in quantum machine learning,” *Phys. Rev. A* **106**, 042407 (2022).
- [6] M. Cerezo et al., “Variational quantum algorithms,” *Nature Reviews Physics* **3**, 625 (2021). A review of the whole family.
- [7] IBM Quantum Learning, *Quantum machine learning*. <https://quantum.cloud.ibm.com/learning/en/courses/quantum-machine-learning>